

Instalación en Ubuntu:

```
sudo apt install docker.io
```

Descarga de una imagen:

```
sudo docker pull hello-world
```

Ejecución de una imagen:

```
sudo docker run hello-world
```

Uso de docker sin ser root. se añade el usuario de Ubuntu dentro del grupo docker

```
sudo usermod -aG docker pc-profe
```

 (hay que cerrar la sesión y volver a entrar para que funcione; si no va, reiniciar Ubuntu)**Versión de docker instalada**

```
sudo docker version
```

Información sobre docker

```
sudo docker info
```

Búsqueda de una imagen en hub.docker.com

```
docker search httpd
```

Búsqueda con filtro:

```
docker search ubuntu:24.04
```

Descargamos httpd y ubuntu

```
docker pull httpd
```

```
docker pull ubuntu
```

Ver las imágenes descargadas

```
docker images
```

Ver los contenedores en ejecución

```
docker ps -a
```

Si se ejecuta una imagen sin más, el contenedor puede arrancar y terminar sin más:

```
docker run ubuntu
```

Para hacer un comando con ubuntu

```
docker run ubuntu ls
```

Lo hace y termina.

Para interactuar con un contenedor, se usan las opciones -it

`docker run -it ubuntu bash`

Aparecerá el identificador del contenedor:

```
pc-profe@pc-77:~$ docker run -it ubuntu bash
root@a5196e5ded11:/#
```

Con otra terminal podemos hacer

`docker ps -a` y ver los estados de los contenedores que se han ejecutado hasta el momento:

```
pc-profe@pc-77:~$ docker ps -a
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a5196e5ded11	ubuntu	"bash"	About a minute ago	Up About a minute		great_beaver
3be2bff9a67b	ubuntu	"ls"	2 minutes ago	Exited (0) 2 minutes ago		intelligent_mayer
c7a12f769eca	ubuntu	"/bin/bash"	2 minutes ago	Exited (0) 2 minutes ago		crazy_chatelet
cf29d855f8d3	hello-world	"/hello"	26 minutes ago	Exited (0) 26 minutes ago		compassionate_hertz

```
pc-profe@pc-77:~$
```

Si se ejecuta un comando dentro del contenedor, como `mkdir pruebas`, lo hace, pero cuando termine el contenedor no recordará nada (no es persistente)

```
pc-profe@pc-77:~$ docker run -it ubuntu bash
root@a5196e5ded11:/# mkdir pruebas
root@a5196e5ded11:/# ls
bin  dev  home  lib64  mnt  proc  root  sbin  sys  usr
boot  etc  lib  media  opt  pruebas  run  srv  tmp  var
root@a5196e5ded11:/#
```

Salida de un contenedor:

`exit`

Borrado de un contenedor de la lista que sale con `docker ps -a`

`docker rm id_contenedor` Ej: `docker rm a5`

```

root@a5196e5ded11:/# exit
exit
pc-profe@pc-77:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS              PORTS          NAMES
a5196e5ded11   ubuntu   "bash"                   7 minutes ago   Exited (127) 10 seconds ago           great_be
aver
3be2bff9a67b   ubuntu   "ls"                     7 minutes ago   Exited (0) 7 minutes ago             intellig
ent_mayer
c7a12f769eca   ubuntu   "/bin/bash"              8 minutes ago   Exited (0) 8 minutes ago             crazy_ch
atelet
cf29d855f8d3   hello-world "/hello"                 32 minutes ago   Exited (0) 32 minutes ago           compassi
onate_hertz
pc-profe@pc-77:~$ docker rm a5
a5
pc-profe@pc-77:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS              PORTS          NAMES
3be2bff9a67b   ubuntu   "ls"                     8 minutes ago   Exited (0) 8 minutes ago             intelligen
t_mayer
c7a12f769eca   ubuntu   "/bin/bash"              9 minutes ago   Exited (0) 9 minutes ago             crazy_chat
elet
cf29d855f8d3   hello-world "/hello"                 33 minutes ago   Exited (0) 33 minutes ago           compassion
ate_hertz
pc-profe@pc-77:~$

```

En vez de 2 terminales se puede usar una con docker y con un contenedor en ejecución, pulsar Control+q+p para ir del contenedor a terminal sin más. Para volver al contenedor, se pone

docker attach id

```

root@830afe11af9f:/# ls
bin boot dev etc home lib lib64 media mnt opt proc root run sbin srv sys tmp usr var
root@830afe11af9f:/# clearpc-profe@pc-77:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS              PORTS          NAMES
830afe11af9f   ubuntu   "bash"                   28 seconds ago   Up 27 seconds           pedantic_f
araday
3be2bff9a67b   ubuntu   "ls"                     12 minutes ago   Exited (0) 12 minutes ago             intelligen
t_mayer
c7a12f769eca   ubuntu   "/bin/bash"              13 minutes ago   Exited (0) 13 minutes ago             crazy_chat
elet
cf29d855f8d3   hello-world "/hello"                 37 minutes ago   Exited (0) 37 minutes ago           compassion
ate_hertz
pc-profe@pc-77:~$ docker attach 83
root@830afe11af9f:/#

```

Observa que basta, en la mayoría de los casos, usar los 2 primeros dígitos del contenedor para referirse a él.

Con los comandos stop, start y attach es posible parar el contenedor, iniciarlo y volver a él.

Hasta que no se crea una imagen de un contenedor, con commit, no se conserva (persiste) lo hecho en el contenedor.

- Se ejecuta el contenedor y se da a apt get update

```
docker run -it ubuntu bash
```

```
apt update
```

```
mkdir prueba
```

```
mkdir 2daw
```

```
19 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@18566a24b095:/# mkdir prueba
root@18566a24b095:/# mkdir 2daw
root@18566a24b095:/# ls
2daw  boot  etc  lib  media  opt  prueba  run  srv  tmp  var
bin   dev  home  lib64  mnt  proc  root  sbin  sys  usr
root@18566a24b095:/#
```

- En otra terminal o en esa terminal con Control+q+p hacemos:

```
docker commit id nombre
```

En nuestro caso:

```
docker commit 18 ubuntu-prueba
```

```
pc-profe@pc-77:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS   NAMES
a5196e5ded11  ubuntu   "bash"    About a minute ago    Up About a minute   great_beaver
3be2bff9a67b  ubuntu   "ls"      2 minutes ago         Exited (0) 2 minutes ago    intelligent_mayer
c7a12f769eca  ubuntu   "/bin/bash"  2 minutes ago         Exited (0) 2 minutes ago    crazy_chatelet
cf29d855f8d3  hello-world  "/hello"    26 minutes ago        Exited (0) 26 minutes ago    compassionate_hertz
pc-profe@pc-77:~$ docker commit 18 ubuntu-prueba
sha256:e90cadb403c55f43bdc0ec89cb2e8b4c0e177636f0147fa912fbb0632b8b718d
pc-profe@pc-77:~$
```

Con docker images se verá que se ha creado

```
pc-profe@pc-77:~$ docker commit 18 ubuntu-prueba
sha256:e90cadb403c55f43bdc0ec89cb2e8b4c0e177636f0147fa912fbb0632b8b718d
pc-profe@pc-77:~$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
ubuntu-prueba latest    e90cadb403c5   5 minutes ago  123MB
httpd         latest    f7d8bafbd9a9   6 days ago     148MB
hello-world   latest    74cc54e27dc4   8 days ago     10.1kB
ubuntu        latest    b1d9df8ab815   2 months ago   78.1MB
pc-profe@pc-77:~$
```

Con esta imagen creada, si la ejecutamos, comprobaremos que se han mantenido las carpetas creadas:

```
pc-profe@pc-77:~$ docker run -it ubuntu-prueba bash
root@79ee889f56ce:/# ls
zdaw bin boot dev etc home lib lib64 media mnt opt proc prueba root run sbin srv sys tmp usr var
root@79ee889f56ce:/#
```

Supongamos que, a partir de este contenedor, se quiere instalar un servidor web Apache:

```
apt update
```

```
apt install apache2
```

```
service apache2 start
```

```
service apache2 status
```

```
root@79ee889f56ce:/# service apache2 start
* Starting Apache httpd web server apache2
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, using 172.17.0.3. Set the 'ServerName' directive globally to suppress this message
*
root@79ee889f56ce:/# service apache2 status
* apache2 is running
root@79ee889f56ce:/#
```

Una vez instalado apache2, se va a crear una imagen, esta vez con una serie de parámetros.

```
docker commit --change='CMD ["apachectl", "-DFOREGROUND"]' -c "EXPOSE 80" 79 apache2mio
```

Ojo: no formar el comando desde un procesador de textos no limpio como Writer, que puede alterar el tipo de comillas y no funcionar correctamente

```
pc-profe@pc-77:~$ docker commit --change='CMD ["apachectl", "-DFOREGROUND"]' -c "EXPOSE 80" 79 apache2mio
sha256:f9360b3ddac4ba3c06ca03d91ebf42fd71bf105e145f75da5b8225a774f55483
pc-profe@pc-77:~$
```

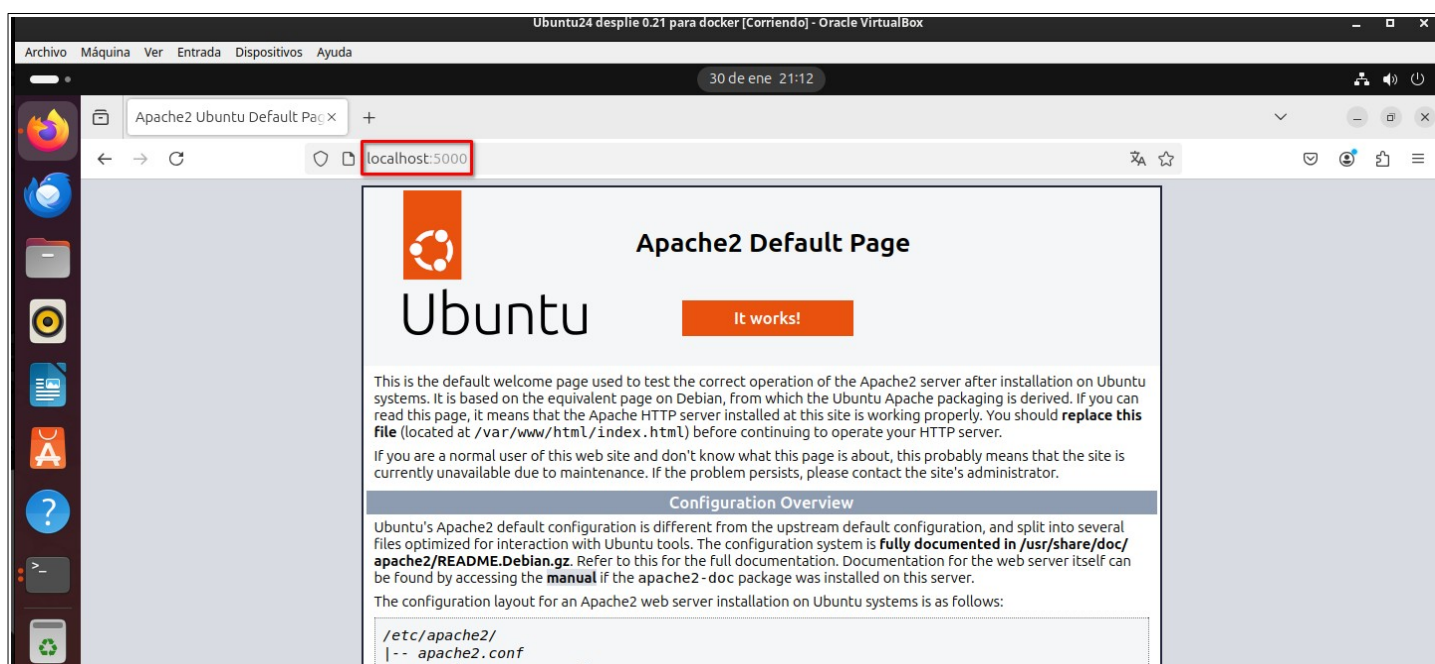
Se ha creado una imagen, llamada apache2mio

```
pc-profe@pc-77:~$ docker images
REPOSITORY          TAG             IMAGE ID         CREATED          SIZE
apache2mio          latest         f9360b3ddac4    5 minutes ago   238MB
ubuntu-prueba       latest        e90cadb403c5    38 minutes ago  123MB
httpd               latest        f7d8bafbd9a9    6 days ago     148MB
hello-world         latest        74cc54e27dc4    8 days ago     10.1kB
ubuntu              latest        b1d9df8ab815    2 months ago   78.1MB
pc-profe@pc-77:~$
```

A continuación vamos a ejecutar la imagen recién creada, iniciando un contenedor con parámetros -d (demonio, para ejecutar en segundo plano) y -p, para asociar el puerto 5000 de nuestro ordenador anfitrión Ubuntu con el puerto 80 del contenedor, donde estará escuchando apache2.

```
pc-profe@pc-77:~$ docker run -d -p 5000:80 apache2mio
07b83169ec98069fed58fa673e45e6dd999d6438ccea47d0a00367faafb4781
pc-profe@pc-77:~$
```

Ahora, si en nuestro sistema operativo anfitrión entramos en el navegador de internet y ponemos localhost:5000, debe aparecer la página web de inicio del servidor apache2 de nuestro contenedor apache2mio



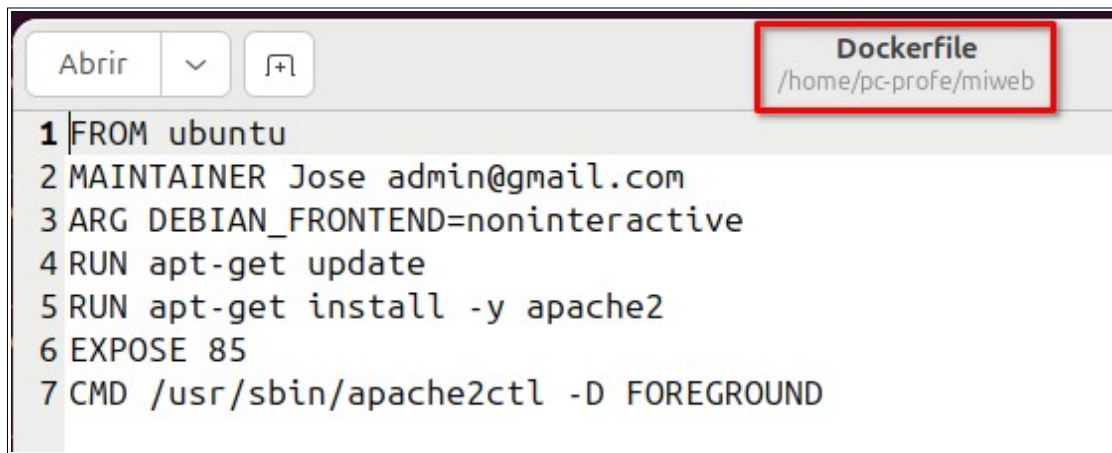
Con

`docker stop id_contenedor` se parará el contenedor. En mi caso es `docker stop 07`

```
pc-profe@pc-77:~$ docker stop 07
pc-profe@pc-77:~$
```


DockerFile sirve para realizar imágenes a partir de un contenedor personalizado de una manera sencilla. Se trata de un documento de texto que contiene los comandos a ejecutar para crear una imagen personalizada. Ejemplo 1:

- Se crea una carpeta llamada miweb dentro de nuestra carpeta personal en Ubuntu.
- Dentro de esa carpeta creamos con un editor de textos un archivo llamado Dockerfile (se tiene que llamar así) con esta información:



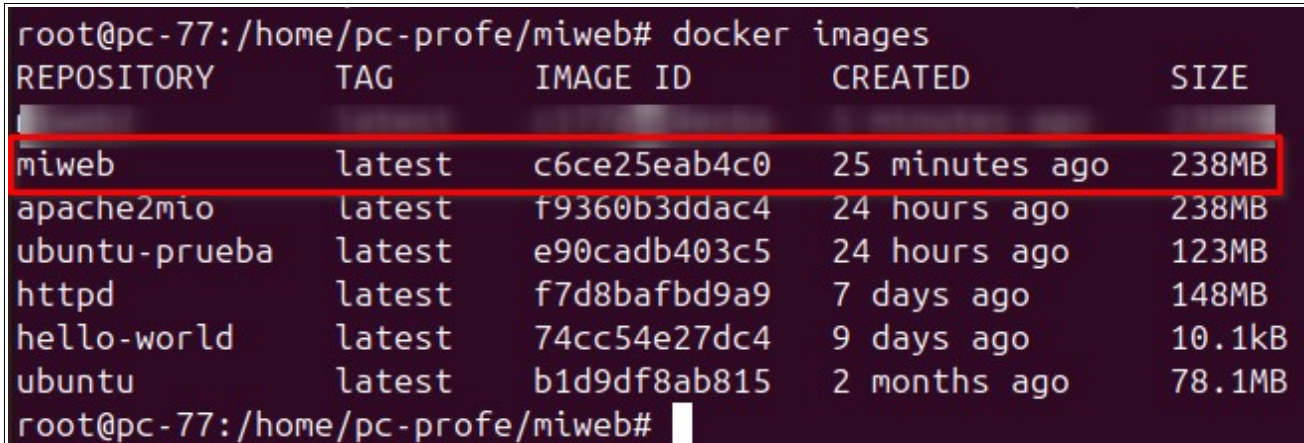
```

1 FROM ubuntu
2 MAINTAINER Jose admin@gmail.com
3 ARG DEBIAN_FRONTEND=noninteractive
4 RUN apt-get update
5 RUN apt-get install -y apache2
6 EXPOSE 85
7 CMD /usr/sbin/apache2ctl -D FOREGROUND
  
```

- Se crea la imagen, llamada miweb, con este comando

```
docker build -t miweb /home/pc-profe/miweb
```

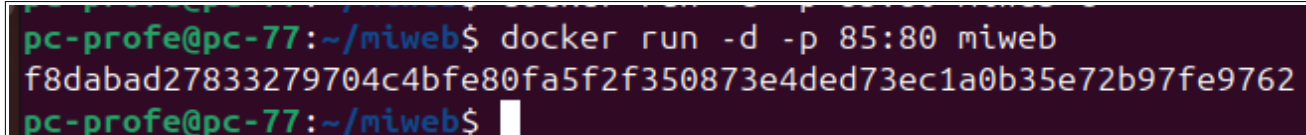
- Se ve que se ha creado satisfactoriamente con **docker images**



```

root@pc-77:/home/pc-profe/miweb# docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
miweb                latest              c6ce25eab4c0       25 minutes ago     238MB
apache2mio           latest              f9360b3ddac4       24 hours ago       238MB
ubuntu-prueba        latest              e90cadb403c5       24 hours ago       123MB
httpd                latest              f7d8bafbd9a9       7 days ago         148MB
hello-world          latest              74cc54e27dc4       9 days ago         10.1kB
ubuntu               latest              b1d9df8ab815       2 months ago       78.1MB
root@pc-77:/home/pc-profe/miweb#
  
```

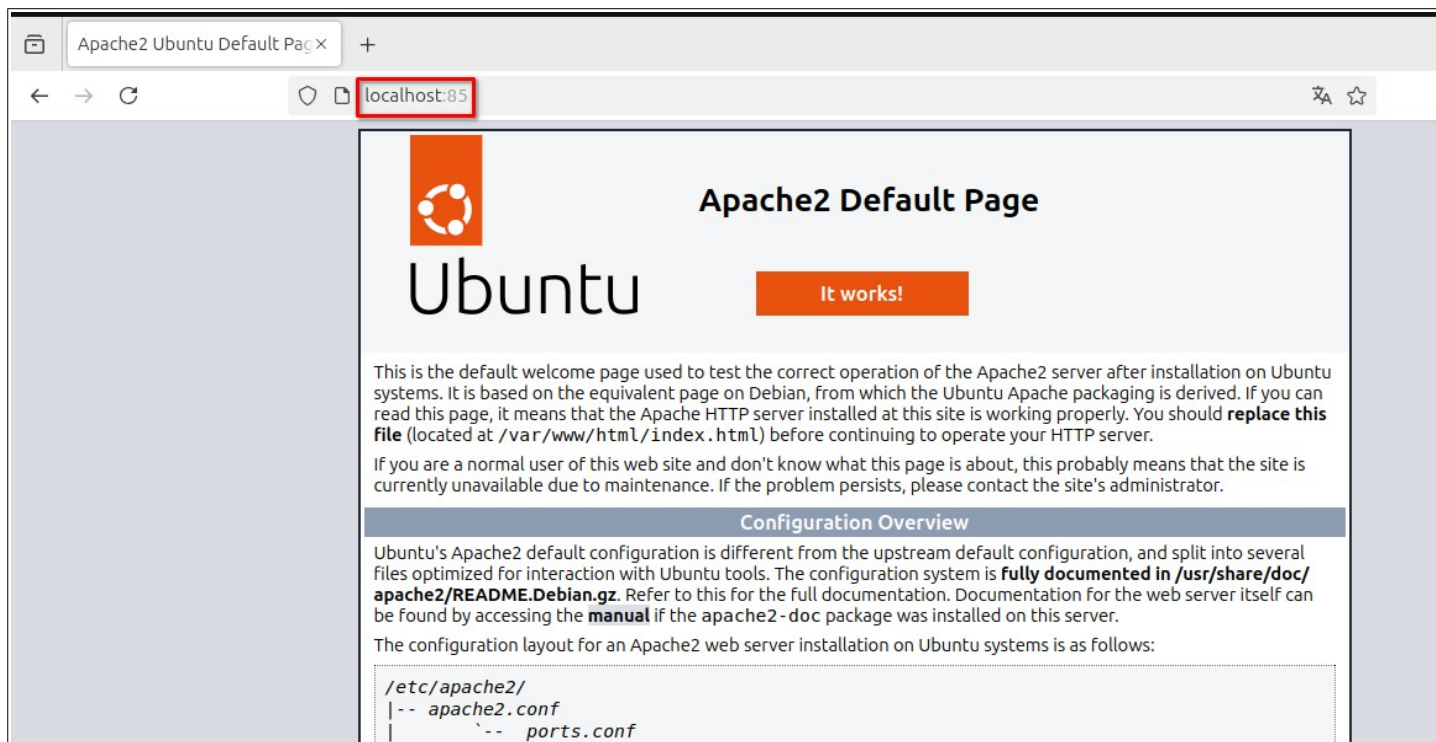
- Ejecutamos la imagen miweb con este comando **docker run -d -p 85:80 miweb**



```

pc-profe@pc-77:~/miweb$ docker run -d -p 85:80 miweb
f8dabad27833279704c4bfe80fa5f2f350873e4ded73ec1a0b35e72b97fe9762
pc-profe@pc-77:~/miweb$
  
```

- Si ahora entramos en el navegador de nuestro sistema operativo anfitrión y escribimos localhost:85 aparecerá la página de Apache:



¡Ha funcionado!

PROBLEMA IMPORTANTE EN VERSIONES FUTURAS DE DOCKER:

Si te has fijado, antes de crear la imagen, se avisa de que esta forma de construir una imagen está descontinuada (deprecated), y que será eliminada esta forma de trabajar en futuras versiones. Por ello vamos a instalar la versión oficial de Docker y a usar el método nuevo que recomiendan:

```
sudo apt remove docker.io # Elimina la versión antigua
sudo apt update
sudo apt install ca-certificates curl gnupg

# Agregar la clave GPG oficial de Docker
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo tee /etc/apt/keyrings/docker.asc > /dev/null
sudo chmod a+r /etc/apt/keyrings/docker.asc

# Agregar el repositorio oficial de Docker
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
  https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
```

Con esto se elimina la versión no oficial que se instaló al principio del tutorial, docker.io, y se instala la última versión oficial de Docker.

Ahora hay que iniciar el servidor Docker, con

```
service docker start
service docker status
```



```

pc-profe@pc-77:~/miweb$ service docker status
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Fri 2025-01-31 20:33:46 CET; 17min ago
 TriggeredBy: ● docker.socket
    Docs: https://docs.docker.com
   Main PID: 13377 (dockerd)
     Tasks: 31
    Memory: 29.9M (peak: 50.5M)
       CPU: 1.338s
    CGroup: /system.slice/docker.service
            └─13377 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
              └─17353 /usr/bin/docker-proxy -proto tcp -host-ip 0.0.0.0 -host-port 85 -container-ip 172.17.0.2 -co
                └─17359 /usr/bin/docker-proxy -proto tcp -host-ip :: -host-port 85 -container-ip 172.17.0.2 -contain>

ene 31 20:33:46 pc-77 dockerd[13377]: time="2025-01-31T20:33:46.159505186+01:00" level=warning msg="error locatin>
ene 31 20:33:46 pc-77 dockerd[13377]: time="2025-01-31T20:33:46.159633712+01:00" level=info msg="Loading containe>
ene 31 20:33:46 pc-77 dockerd[13377]: time="2025-01-31T20:33:46.178547093+01:00" level=info msg="Docker daemon" c>
ene 31 20:33:46 pc-77 dockerd[13377]: time="2025-01-31T20:33:46.178597135+01:00" level=info msg="Daemon has compl>
ene 31 20:33:46 pc-77 dockerd[13377]: time="2025-01-31T20:33:46.193458329+01:00" level=info msg="API listen on /r>
ene 31 20:33:46 pc-77 systemd[1]: Started docker.service - Docker Application Container Engine.
ene 31 20:33:55 pc-77 dockerd[13377]: time="2025-01-31T20:33:55.747981935+01:00" level=info msg="No non-localhost>
ene 31 20:33:55 pc-77 dockerd[13377]: time="2025-01-31T20:33:55.768396882+01:00" level=info msg="No non-localhost>
ene 31 20:33:58 pc-77 dockerd[13377]: time="2025-01-31T20:33:58.435423535+01:00" level=info msg="No non-localhost>
lines 1-23
pc-profe@pc-77:~/miweb$

```

Por último, se ejecuta el comando para crear con el nuevo método la imagen, que llamaremos miweb2

docker buildx build -t miweb2 /home/pc-profe/miweb/ --load

Ha funcionado, se ve con **docker images**

```

pc-profe@pc-77:~/miweb$ docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
miweb2              latest         c177a914ecba   19 minutes ago 238MB
miweb               latest         c177a914ecba   19 minutes ago 238MB
<none>              <none>         c6ce25eab4c0   40 minutes ago 238MB
apache2mio          latest         f9360b3ddac4   24 hours ago   238MB
ubuntu-prueba       latest         e90cadb403c5   24 hours ago   123MB
httpd               latest         f7d8bafbd9a9   7 days ago     148MB
hello-world         latest         74cc54e27dc4   9 days ago     10.1kB
ubuntu              latest         b1d9df8ab815   2 months ago   78.1MB
pc-profe@pc-77:~/miweb$

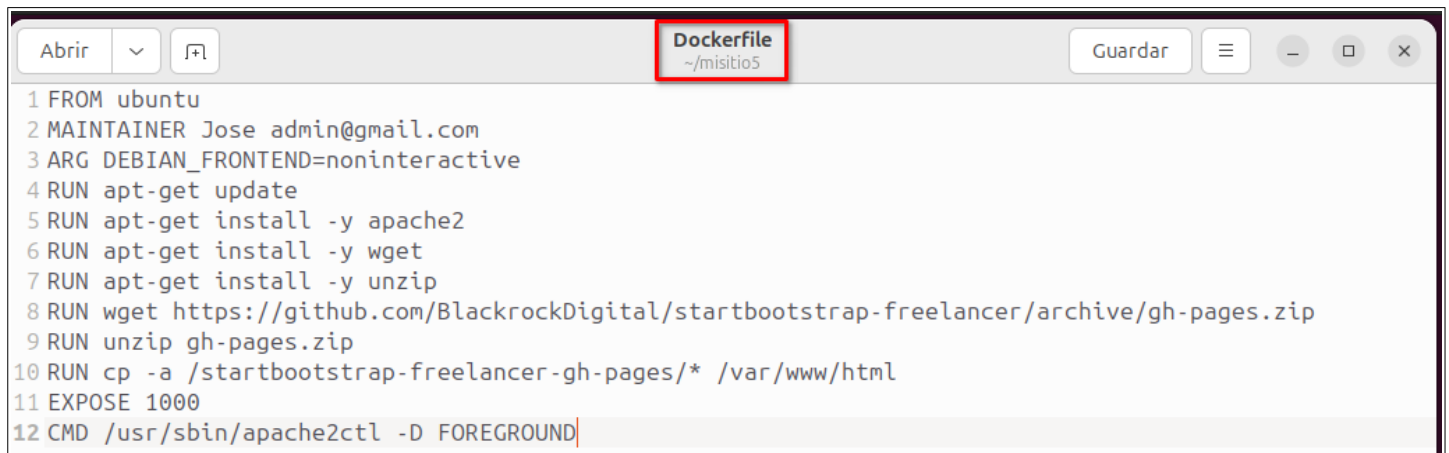
```

Ejemplo 2:

Se descarga a través de Github una web, se descomprime y se copia a la ruta por defecto de apache2mio

Creamos una carpeta llamada misitio5 dentro de nuestra carpeta personal.

Creamos un archivo Dockerfile con esta información:

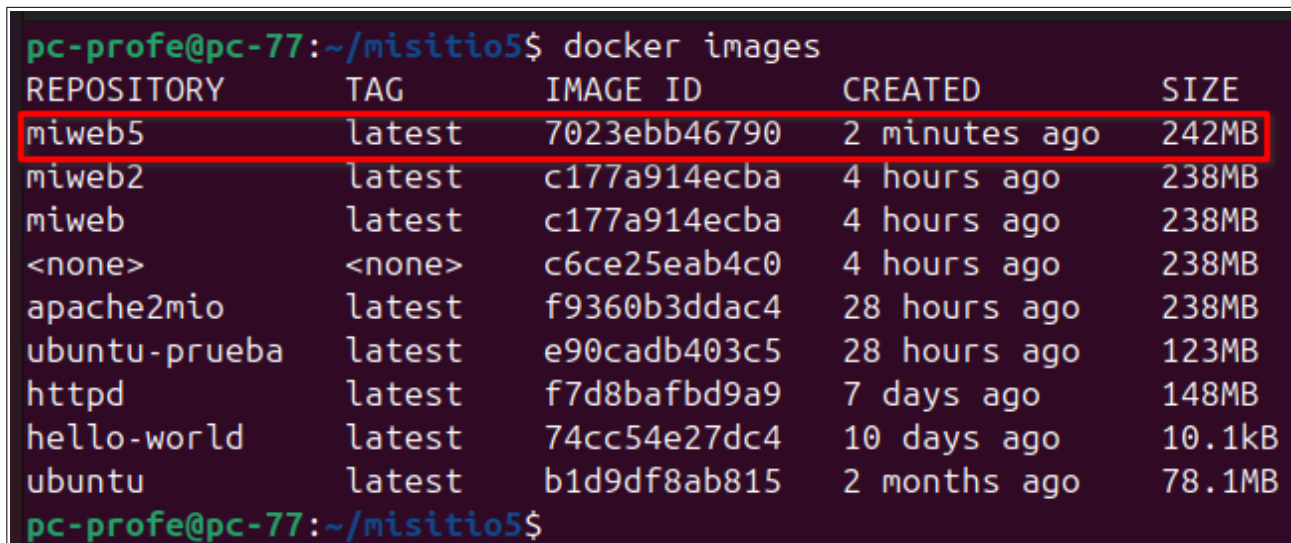


```
1 FROM ubuntu
2 MAINTAINER Jose admin@gmail.com
3 ARG DEBIAN_FRONTEND=noninteractive
4 RUN apt-get update
5 RUN apt-get install -y apache2
6 RUN apt-get install -y wget
7 RUN apt-get install -y unzip
8 RUN wget https://github.com/BlackrockDigital/startbootstrap-freelancer/archive/gh-pages.zip
9 RUN unzip gh-pages.zip
10 RUN cp -a /startbootstrap-freelancer-gh-pages/* /var/www/html
11 EXPOSE 1000
12 CMD /usr/sbin/apache2ctl -D FOREGROUND
```

Usamos el comando

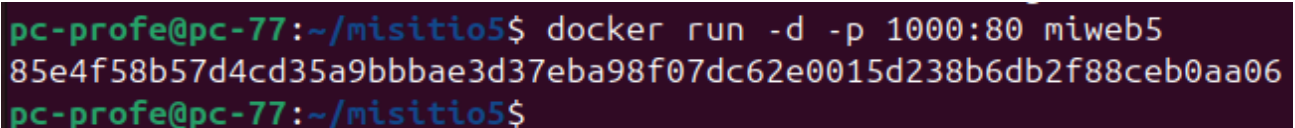
docker buildx build -t miweb5 /home/pc-profe/misitio5/ --load

Ha funcionado, se ve con el comando **docker images**



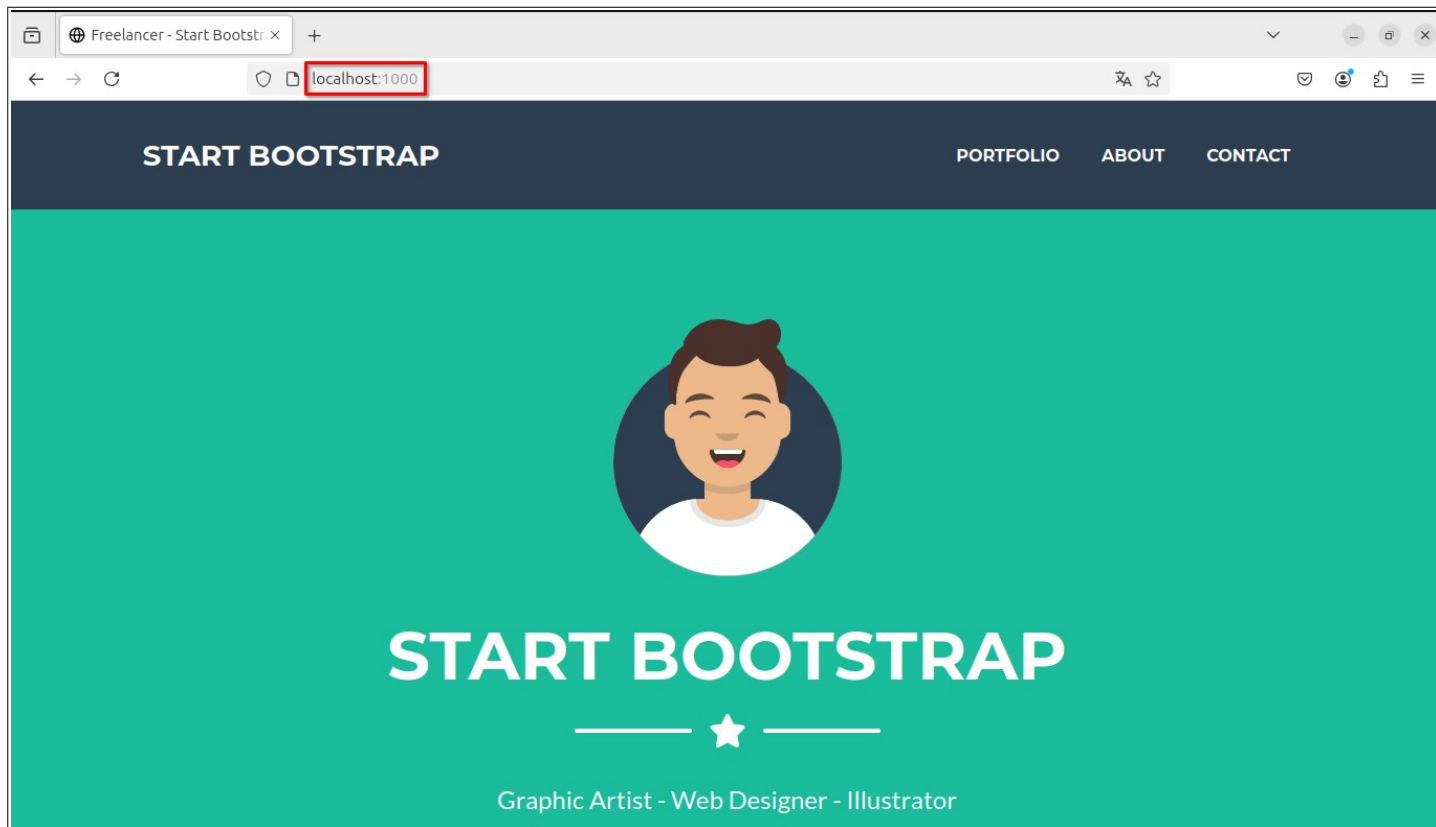
```
pc-profe@pc-77:~/misitio5$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             SIZE
miweb5              latest             7023ebb46790       2 minutes ago      242MB
miweb2              latest             c177a914ecba       4 hours ago        238MB
miweb               latest             c177a914ecba       4 hours ago        238MB
<none>              <none>             c6ce25eab4c0       4 hours ago        238MB
apache2mio          latest             f9360b3ddac4       28 hours ago       238MB
ubuntu-prueba       latest             e90cadb403c5       28 hours ago       123MB
httpd               latest             f7d8bafbd9a9       7 days ago         148MB
hello-world         latest             74cc54e27dc4       10 days ago        10.1kB
ubuntu              latest             b1d9df8ab815       2 months ago       78.1MB
pc-profe@pc-77:~/misitio5$
```

Vamos a crear un contenedor de esa imagen, para ver si funciona:



```
pc-profe@pc-77:~/misitio5$ docker run -d -p 1000:80 miweb5
85e4f58b57d4cd35a9bbbae3d37eba98f07dc62e0015d238b6db2f88ceb0aa06
pc-profe@pc-77:~/misitio5$
```

Entramos en el navegador de nuestro sistema operativo anfitrión y escribimos localhost:1000



¡¡Funciona!!

MEJORAS

MEJORA 1:

Al ejecutar la instrucción

docker buildx build -t miweb5 /home/pc-profe/misitio5/ --load

sale un aviso de este tipo:

MaintainerDeprecated: Maintainer instruction is deprecated in favor of using label (line 2)

Para evitarlo, hay que

1) Cambiar la línea 2 de Dockerfile, sustituyendo **MAINTAINER Jose admin@gmail.com** por

LABEL author="Jose" email="admin@gmail.com" version="1.0"



2) Usar esta orden para crear la imagen:

```
docker buildx build -t miweb5 /home/pc-profe/miweb/ --load --no-cache
```

MEJORA 2:

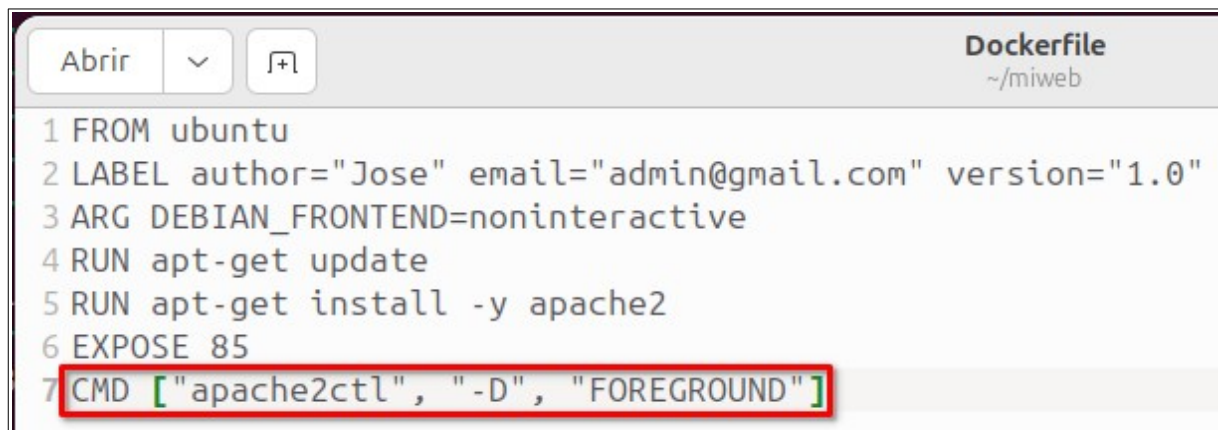
Para evitar este aviso en la generación de la imagen

1 warning found (use docker --debug to expand): - JSONArgsRecommended: JSON arguments recommended for CMD to prevent unintended behavior related to OS signals (line 7)

hay que cambiar la línea 7 de Dockerfile por

`CMD ["apache2ctl", "-D", "FOREGROUND"]`

El archivo quedará así:

A screenshot of a code editor window titled "Dockerfile" with a subtitle "~/miweb". The editor contains a Dockerfile with seven lines of code. Line 7, which is `CMD ["apache2ctl", "-D", "FOREGROUND"]`, is highlighted with a red rectangular box. The other lines are: 1 FROM ubuntu, 2 LABEL author="Jose" email="admin@gmail.com" version="1.0", 3 ARG DEBIAN_FRONTEND=noninteractive, 4 RUN apt-get update, 5 RUN apt-get install -y apache2, and 6 EXPOSE 85.

```
1 FROM ubuntu
2 LABEL author="Jose" email="admin@gmail.com" version="1.0"
3 ARG DEBIAN_FRONTEND=noninteractive
4 RUN apt-get update
5 RUN apt-get install -y apache2
6 EXPOSE 85
7 CMD ["apache2ctl", "-D", "FOREGROUND"]
```

Se ejecutará el comando de generación de imagen

```
docker buildx build -t miweb2 /home/pc-profe/miweb/ --load --no-cache
```

y ya no aparecerán más avisos.